VRIJE
UNIVERSITEIT
BRUSSEL

Graduation thesis submitted in partial fulfilment of the requirements for the degree of de Ingenieurswetenschappen: Computerwetenschappen

PROGRAMMING ROBOTS WITH HIGHER-ORDER REACTIVE PROGRAMMING

Gérard Lichtert

2025-2026

Promotor: Prof. Dr. Wolfgang De Meuter
Advisor: Bjarno Oeyen

Sciences and Bio-engineering Sciences

VRIJE
UNIVERSITEIT
BRUSSEL

Proefschrift ingediend met het oog op het behalen van de graad van Master of Science in de Ingenieurswetenschappen: Computerwetenschappen

ROBOTS PROGRAMMEREN MET HOGERE-ORDE REACTIEF PROGRAMMEREN

Gérard Lichtert

2025-2026

Promotor: Prof. Dr. Wolfgang De Meuter
Advisor: Bjarno Oeyen

Wetenschappen en Bio-ingenieurswetenschappen

Abstract

Reactive programming (RP) is a declarative programming paradigm based on data streams and the propagation of change. In a nutshell, if we have two variables x and y and another variable z that sums both x and y, it should be sufficient to update either x or y and z should be updated automatically.

There are several approaches to achieve reactive programming. One approach is to create a dedicated language. In this case the reactivity will be native to the language. Another option is to extend a language with reactive features, like

Contents

Chapter 1

Introduction

Modern software is often written in such a way that it is event-driven, or in other words, reactive. This means that applications reacts to an incoming event by creating subsequent events that arise as a reaction to the incoming event. Such an application is often called a reactive system. Reactive systems are often implemented with callbacks or by implementing the Observer Pattern. However, this does not come without its drawbacks as this may lead to 'inversion of control' or 'callback hell'[1], [2]. To alleviate this we introduce **Reactive Programming**. A programming paradigm that allows the user to write reactive systems in a declarative way. Traditionally the sum and the product of two elements would be written as shown in Listing 1.

```
int x = 5;
int y = 10;
int s = x + y;
int p = x * y;
// x changes state
x = 10;
// s requires recomputation
s = x + y;
// p requires recomputation
p = x * y;

```

Listing 1: Imperative product and sum

Whenever x or y would change state, you would have to manually update s and p. Reactive programming allows you to declare x and y as sources of data and s and p as the result of the relevant computations. Whenever the value of x or y would change, the variables dependent on the sources would be automatically recomputed, and the result propagated further in the reactive system. Schematically the dependency graph of product-sum would look like Figure 1.1

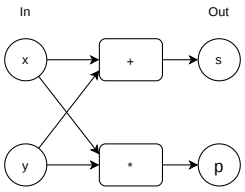

Figure 1.1: Dependency graph for product-sum

There are different ways of how such a reactive system could be created using reactive programming. One way is to use a programming which itself is natively reactive. A few examples are Haai [3], Another is to add native reactivity to an existing language by extending it the language or by implementing a reactive library such as ReactiveX. We will be looking at the former and more specifically the higher-order reactive programming language **Haai**, which was developed at the SOFT lab at the VUB. Haai uses reactors as their As a preview, an example of how such a product and sum could be written in Haai is shown in Listing 2. Haai uses reactors as its native way of declaring a reactive program.

```
(defr (product-sum x y)
    (def sum (+ x y))
    (def product (* x y))
    (out product sum))

```

Listing 2: Product sum in Haai

Additionally, we want to delve into the world of electronics. From maintaining your fridge's temperature to controlling a drones' altitude, electronics are deeply embedded in everyone's day-to-day life. Writing software for these electronics can also be challenging. This mainly due to the fact that using peripherals require using their associated libraries, which are not always standardized. Another challenge is that creating programs for microcontrollers often require the developer to use a low-level programming language such as C/C++ or Rust, which come with their own challenges. Nowadays, there are some efforts being made to provide a standardized collection of tools to aid with the development of systems with many peripherals, or robots. There is Robot Operating Systems (ROS) [4], Open Robot Control Software (OROCOS) [5], Yet Another Robot Platform (YARP) [6] to name a few. We will be focussing on using **(ROS)**.

1.1 Contributions

Reactive programming is used across a multitude of domains. It is used in web and mobile development for instance. However, we will be focussing on using it for the Internet of Things (IoT). IoT consists of multiple physical objects, usually with very limited computing power and/or memory. In this thesis we introduce a way of declaring a reactive program across multiple devices from a single source. This means that the developer declares the program once,

along with declarations where which part of the program lives and then deploys the program to the relevant devices making use of ROS. This is achieved by the following components:

- An extension of the Remus VM [7], more specifically, extending the language to accept a declaration of where which part of the program lives. The Remus compiler will use this metadata to split the Haai reactor declarations into an embedded part and a server part. Each piece will then be available for the user to be deployed in their respective parts of the IoT network.
- A new tool called CHaai, which translates the Remus bytecode obtained from compiling the Haai program into a C program. The C program will be required to run the program on microcontrollers using ROS.
- A C port of the existing Remus VM written in rust [8], also created to allow developers to create reactive programs for microcontrollers. However, it is adapted with respects to the current Instruction Set Architecture (ISA) of the Racket version of Remus [7] and allows usage of ROS.

1.2 Overview of this Dissertation

Chapter 2 gives the problem statement as well as an overview of the current reactive programming solutions and existing toolkits to program networks of microcontrollers. It also introduces a concept called 'Tierless programming', which will inspire the declarative way of telling Remus where which part of the reactive program lives.

Chapter 3 introduces μ-Remus. A C port of the Remus VM, which was previously written in Rust as well as in Racket. It explains why the C port was necessary as well as conceptualizes the link between the ROS communication protocols and the dependency graph generated by reactive programming.

Chapter 4 demonstrates usage of the Haai language extensions inspired by Tierless programming to declare where which part of the reactive program lives. It will also tell you about the CHaai tool. Which will create a C version of the reactive program. It is ported to C to be compatible with ROS.

Chapter 5 will evaluate the tools and extensions driven by an example. The target microcontroller will be a Raspberry Pi Pico H (2021) and will be used along a PC.

Lastly, chapter 6 will conclude this dissertation and give notes on some possible future work.

1.3 Approach

We first start by discussing the state of the art, what tools already exist and then move on to the problem statement and continue with the contributions part of the dissertation. However,

since it is heavily dependent on a language and a toolkit we will first give a brief introduction into some relevant parts of the language or toolkit, each time providing the necessary context and information, prior to moving on to the actual contributions.

Chapter 2

State of the Art

Different programming languages have adopted various paradigms, including functional or logic models. Yet imperative programming remains a popular paradigm due to how popular languages have been designed. However, as applications shifted towards event-driven architectures, traditional imperative programming did not come without its drawbacks. Traditional imperative programming was simple. Each imperative program was code that defined its step-by-step control flow and state transitions. However, when an imperative programming language is used for an event-driven architecture, some problems may arise, such as 'inversion of control' or 'callback hell'. Inversion of control is a problem commonly associated with codebases using frameworks, where instead of the programmer calling functions provided by the framework, the framework calls the functions defined by the programmer.

```
func1(argument0, function(argument1){
    // compute result1
    func2(result1, function(argument2){
        // compute result2
        func3(result2, function(argument3){
            // compute result3
            func4(result3, function(argument4){
                //etc..
            })
        })
    })
})
```

Listing 3: Illustration of callback hell in JavaScript

Callback hell occurs when programmers are required to pass a callback function as an argument to another function, which calls the passed function with the result of its computation, yet that function also expects a function to be passed to continue its execution. While this is not necessary an issue if the nesting is one or two layers deep, it becomes 'hell' once it goes multiple layers deep or ad infinitum, as shown in Listing 3. Reactive programming seeks to solve these issues.

2.1 Reactive Programming

Reactive programming (RP) is a declarative programming paradigm that allows the developer to declare a program in terms of dependencies, instead of having to manually manage these dependencies. The actual propagation of the state change in the dependency chain is abstracted away in the language design. For instance, in a program you could have two variables x and y that hold values and variables s and p which hold the sum and product of x and y respectively. Under the imperative paradigm, whenever x or y would change state, it would require manual

```
int x = 5;
int y = 10;
int s = x + y;
printf("%d", s) // 15;
x = 10; // state change
printf("%d", s) // 15;
// requires manual recomputation of s
s = x + y;
printf("%d", s) // 20;
```

Listing 4: Manual recomputation of s, a dependent on x

recomputation of s as shown in Listing 4. In reactive programming, it is sufficient to update either x or y and the change is propagated throughout its system automatically, as shown in Listing 5. In essence, it reacts to changes in its input values and automatically propagates it

```
int x = 5;
int y = 10;
int s = x + y;
printf("%d", s) // 15;
x = 10; // state change
// No manual recomputation needed
printf("%d", s) // 20;
```

Listing 5: Automatic recomputation of s, a dependent on x (Pseudocode)

through itself and updates its output values. These values are also called *signals*. There seem to be two main families in how reactive programming is implemented. One of which is functional reactive programming (FRP) and the other graph-based reactive programming. FRP languages such as E-FRP, FRPNow or Yampa are often implemented in a host language, such as, but not limited to: Haskell, Racket, Java, Scala or JavaScript [1]. These FRP languages then inherit the toolchains of the host language. However, there are also FRP languages are that are an independent language. They do not rely on a host language but instead have their own dedicated tools (compiler, interpreter, etc. . .). FRP Languages consist of lambdas or functions that are (re)-applied every turn in the reactive system.

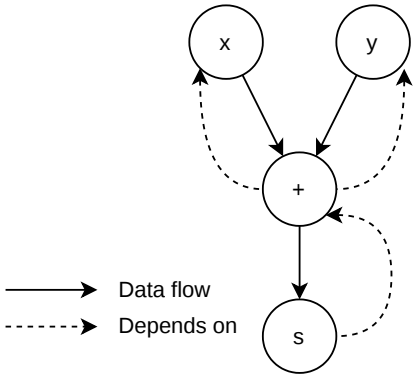

Figure 2.1: DAG for Listing 5

Graph-based reactive programming languages compile the program into a directed acyclic graph (DAG), which is also known as the dependency graph of the program. The DAG describes the data flow between different components within the program, as an example, we can construct the DAG of Listing 5. Since s depends on the result of the sum, and the sum depends on x and y, we can construct the DAG as shown in Figure 2.1. Here we can see that x and y flow to the sum, whose result flows to s. Examples of graph-based reactive programming languages are REScala, FrTime and Flapjax.

A reactive application created with an RP language built on top of a host language usually consists of two parts. A reactive part of code, which is a 'reactive' extension of the host language and a non-reactive part of the code. The non-reactive part of the code are features that are made available from the host language through a mechanism known as *lifting* [3]. However, lifting can lead to issues such a diverging code, which leads to unresponsive programs or a mismatch between the reactive and non-reactive code, which leads to faulty behaviour. The SOFT lab at the VUB proposes a novel reactive programming language called **Haai** [3] (Pronounced 'high') that is meticulously designed to be reactive-only, avoiding these issues completely.

```
1 (defr (sum-product x y)
2     (def s (+ x y))
3     (def p (* x y))
4     (out s p))
```

Listing 6: Reactive program in Haai

Haai uses reactors as its only computational abstraction for declaring a reactive program. Which is a declarative way of defining signals and their dependencies. A Haai program consists of reactor definitions (defined using $defr$) and reactor deployments (similar to application expressions in Racket). An example of such a Haai program can be seen in Listing 6. Here we can see that the reactor reacts to the input signals x and y, and computes their sum and product by deploying the $*$ and $+$ reactor with the input signals and outputs the result of the deployments. Upon changing the values of the input signals, the reactor will automatically recompute their dependant output values.

2.2 RP languages targeting MCUs

In Section 2.1 we discussed reactive programming. Reactive programming has many applications, from web applications that react to the input of users to embedded devices, which react to signals from hardware. In this thesis we want to focus on RP related to embedded devices, whose usage has increased due to their wide variety of applications, such as on the internet of things (IoT), edge computing and artificial intelligence (AI) [9], [10], [11].

As previously mentioned, many RP languages are built on top of host languages which often require the toolchains of those host languages to run. However, in environments where resources are constrained such as on microcontrollers, it is hard to ship an RP program created in Yampa. This is due to the fact that Haskell runtimes are very large and do not fit the resource constraints on microcontrollers. Despite this challenge there have been efforts to get RP onto microcontrollers.

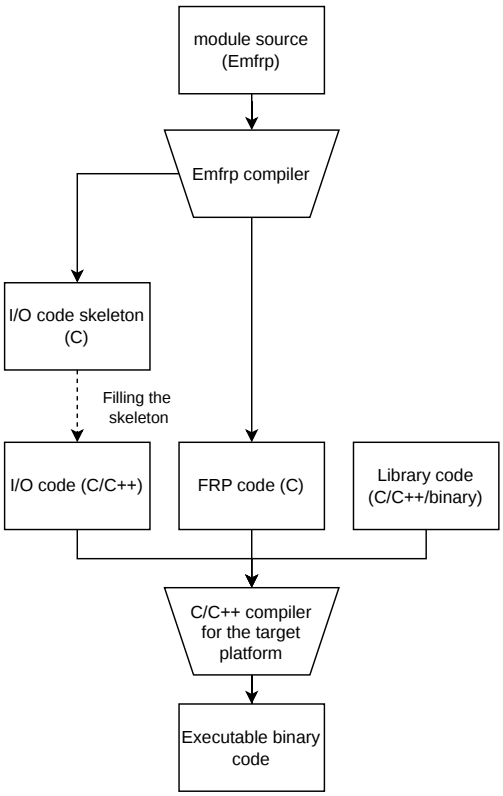

Figure 2.2: EmFRP development workflow [12]

An example would be **EmFRP**, an FRP language created by K. Sawada and T. Watanabe

that mainly targets small-scale embedded systems [12]. Like Haai, EmFRP is a graph-based RP language. This means that EmFRP follows the same philosophy in terms of values that depend on other signals and automatic change propagation. More concretely, a EmFRP program is written in terms of nodes, who may refer to other nodes that it depends on. Conceptually it works the same as the DAG that Haai generated in Figure 2.1.

However, since EmFRP is designed to work on small-scale embedded systems, some design considerations have been made. For instance, the dependency graph must be known statically, meaning it cannot change during runtime. Additionally, it does not allow recursion in function or in type definitions and nodes must always have a name. This because they are not first class values in the language.

```
module Thermostat
in
    temperature : Int,  # temp sensor
    pulse1min : Bool    # periodical pulse
out
    switching : Bool    # heater state
use
    Std                 # standard library

node init [ False ] switching =
    if pulse1min then
        temperature < 50
    else
        switching@last  # previous value
```

Listing 7: Thermostat controller in EmFRP [12]

```
#include "Thermostat.h"
void Input(int *temperature, int *pulse1min){
    /* Your code goes here ... */
}

void Output(int *switching){
    /* Your code goes here ... */
}

int main(){
    ActivateThermostat();
}
```

Listing 8: I/O Code Skeleton for Thermostat [12]

Furthermore, interacting with hardware requires using magic constants such as memory addresses, that are architecture- or peripheral specific. To address this, the EmFRP compiler generates an I/O code skeleton to connect the program with the hardware. The developer remains responsible for completing the skeleton code though.

The way EmFRP works is as follows: The developer creates a series of EmFRP modules, the

EmFRP compiler then generates I/O skeleton code in C and FRP code in C. The developer is then required to fill in the I/O skeleton code. A schematic of the workflow can be seen in Figure 2.2.

In the paper introducing EmFRP, they show an example with a thermostat, where each minute, the temperature is read and depending on the temperature value, the heater is turned on or off. The code for this example can be seen in Listing 7. The EmFRP program is then compiled into the I/O skeleton shown in Listing 8 as well as the FRP code in C. Once the skeleton code is completed, the program can be further compiled by the relevant C compilers into the required binaries.

Another RP language targeting embedded devices is CFRP [13]. By the same creators of EmFRP, K. Sawada, T. Watanabe, as well as K.Suzuki and K. Nagayama, CFRP was created as a pure FRP language that compiles to stand-alone C++ code. It was created to show that FRP is beneficial for developing software for small-scale embedded devices.

```
-- Including C++ header files
%{
#include "sensors.h"
#include "ac_ctrl.h"
%}

-- Declaring identifiers connected to external devices
%input tmp :: Signal Float = sen_tmp;
%input hmd :: Signal Float = sen_hmd;
%import ac :: Signal Bool → Signal () = ctrl_ac;

-- Main expression
let di t h = t - 0.55 * (1 - 0.01 * h) * (t - 14.5) in
ac (lift1 (\x → x > 24) (lift2 di tmp hmd))
```

Listing 9: Example program in CFRP [13]

It was designed as follows: A CFRP program consists of a series of declarations, which denote signals and links to hardware (in C++). Followed by a single expression that builds to reactive system. An example can be seen in Listing 9 where the air conditioner is turned on or off depending on a discomfort index. The discomfort index is calculated from the temperature (tmp) and humidity (hmd). Notably it seems to use the same 'base' datatypes that C++ provides but signals get an additional annotation (the Singal type). This is to make the datatype predictable at compile time.

Furthermore, it uses a single Haskell-like expression to construct a dependency graph, just like EmFRP and Haai do. Which is then used to compile to the C++ program. The caveat with CFRP is that C++ objects and functions referred to in the CFRP file have to be declared in separate C++ files. This means that only the 'global' reactive system is declared in CFRP and the rest in C++.

Whilst there are more RP languages targeting embedded systems like Atom [14] or Copilot [15]. They seem all to compile to a low-level programming language such as C, C++, Rust or Zig. Though the more customary languages that these RP languages compile to are

either C or C++.

At the VUB a virtual machine (VM) was created for Haai to be able to run Haai on the bare metal. The VM is called Remus [7], which was later re-implemented in Rust and with some modifications a prototype was created to run a Haai program on a M5Stick [8]. The benefit of maintaining a scheme/racket-like syntax with dynamic typing remains, at the cost of having to compile the Haai program to Remus bytecode. Which can then be interpreted by the Rust based implementation of Remus.

At this point one could ask him or herself what the point is, of examining these RP languages that target MCUs. Well as mentioned in Section 2.2, due to the increase in use of microcontrollers. Though in this thesis we want to focus more on a system of these embedded devices. Take a robot for example: A robot has sensors as input and has other peripherals that are then used to interact with the real world, depending on these inputs. In other words, the robot reacts to the sensors by instructing the motors what to do or where to move next.

To build this system of embedded devices we could create some reactive programs in an RP language of choice that targets these devices, but this would leave the developer responsible for implementing all the communication between the devices as well as some central computing unit that coordinates all these devices and peripherals. Most of the time requiring the developer to use a library that uses some kind of communication protocol under the hood (Bluetooth, UART, I2C, SPI, USB, etc...). Easier would be if we could use some kind of standardized toolkit that saves the developer from these complexities, which is what we will be looking into next.

2.3 Systems to program robots

In this section we are going to have a look at the following systems that aid development in robotics: Open Robots Control Software (OROCOS) [5], Robot Operating System (ROS) [4], [16], [17], [18] and Yet Another Robot Platform (YARP) [6].

(a) OROCOS logo (b) ROS logo (c) YARP logo

Figure 2.3: Robotics platforms

2.3.1 OROCOS

OROCOS, started at the Katholieke Universiteit Leuven (KUL), as a project aimed at becoming a general-purpose and open robot control software package. On a more technical side of things, it is software for all sorts of robotic devices and computer platforms (operating systems). OROCOS features software components for kinematics, dynamics, planning, sensing, control, interfacing, etc... through a middleware. The system also provides features in the form of modules that can be split up in the following categories:

- Supporting modules

  Software that does not interact with the hardware directly but rather supports the interaction with it. These modules can range from simple mathematical modules to aid vector operations, configuration modules, simulation modules to real-time operating system modules.
- Robotics modules These are the modules specifically interacting with the hardware such as kinematics and dynamics modules or servo controller modules. These robotics modules tend to use one or more of the supporting modules to make the usage or development of these robotics modules easier.
- Components These are the Common Object Request Broker Architecture (CORBA) with their Interface Definition Language (IDL) descriptions. These components are built upon the previous modules and are used as the building blocks of robots, as they allow for multi-machine inter-process communication.

Unfortunately OROCOS does not allow it to be used on the bare metal. Meaning it is only usable on a system that has an OS. However, with the first version of ROS, the real-time toolchain (RTT) of OROCOS, called OROCOS RTT was often used alongside ROS. Nowadays, the second version of ROS, or ROS2 is used in favour of ROS + OROCOS RTT, or the OROCOS Component Library (OCL) as ROS2 absorbed both the toolchain and component features of OROCOS and thus no longer needs them as middleware. The hardware libraries for Kinematics Dynamics (KDL) or Bayesian Filtering (BFL) are still used. OROCOS also provides support to interop with other robotics systems such as ROS and YARP.

2.3.2 ROS

Originating from Stanford, ROS provides a structured communication layer above the operating systems of a compute cluster [16]. A ROS cluster consists of several on-board devices and off-board devices, which can connect and disconnect at runtime. Consequently, the designers of ROS chose for a peer-to-peer (p2p) system as having a central server would mean a lot more network saturation as messages would have to a central point and would then have to be forwarded to their respective recipients. In a p2p system each message would be forwarded to an intermediary recipient who has the address of the final recipient or another intermediary, leading to less network saturation and shorter distances travelled.

The philosophy of ROS of using multiple standalone components within its system allows for developers to reuse or repurpose existing hardware libraries within ROS, but without actually

depending on ROS. This also allows for robust testing of these libraries, without being dependent on ROS. The decoupling of these components also means that they can be used in other robotics systems where there is no inherent dependency and vice versa, giving ROS access to existing standalone components, such as drivers and vision algorithms.

As previously mentioned, the designers of ROS chose for a p2p system. However, the actual implementation is divided into nodes, messages, topics and services. Nodes are processes that perform computations. This means that each device in the system could consist of one or more nodes. Since these nodes also communicate with each other, we could create a graph with the edges denoting where the nodes send their messages to.

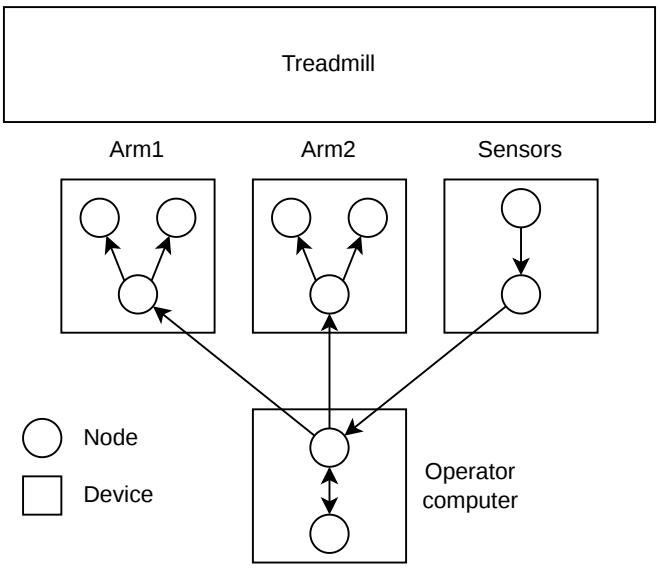

Figure 2.4: ROS node graph example

Figure 2.4 shows a robotics example where some products are being transported down a treadmill. However, these products have to be classified by colour, which the sensors on the 'sensors' device can detect. The sensors can then send the detected colour to the operator computer, which selects the appropriate arm to sort the product. Each arm consists of a series of fingers, which each node controls individually. Furthermore, the operator computer shows the detected colours on the screen and allows the user to manually operate an arm if need be thus completing the graph shown in the figure.

Messages that these nodes send to each other are actually strictly typed datastructures. These can consist of the primitive types (integer, floating point, boolean, etc. . .), arrays of primitive types as well as constants. Messages can also be composed of other messages or arrays of messages which can be nested arbitrarily deep.

To be able to broadcast messages to multiple devices, ROS sends messages through a topic. Devices subscribe to a particular topic if they wish to receive messages published to the topic.

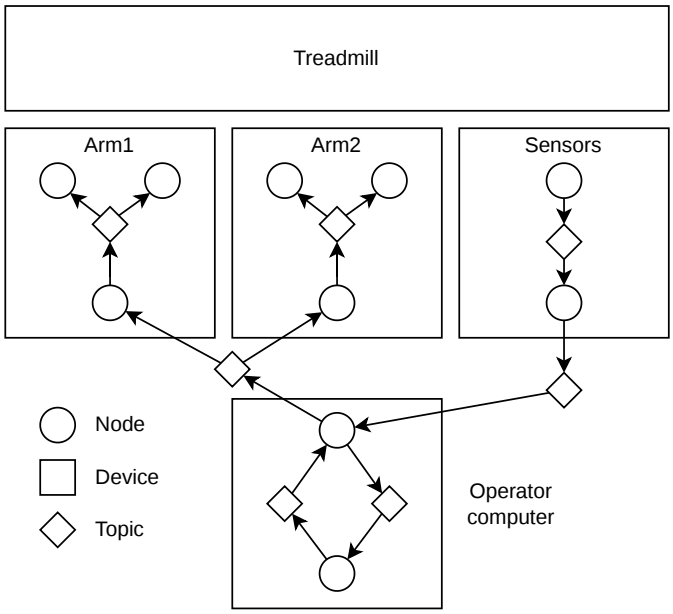

Figure 2.5: ROS node graph with topics

This allows for a many-to-many relationship within the ROS2 node graph. A re-illustration of Figure 2.4 can be seen in Figure 2.5, this time illustrated with the topics, where unidirectional many-to-many relationships are allowed. A bidirectional relationship between two nodes can be created by making one of the nodes a subscriber to a topic and the other a publisher as well as the inverse on another topic.

2.3.3 YARP

2.4 Tierless Programming

2.5 Problem Statement

Chapter 3

μ-Remus

3.1 Haai & Remus

3.2 ROS & μ-ROS

3.3 Mapping RP to ROS & μ-ROS

3.4 μ-Remus: A C implementation

Chapter 4

Programming Robots with Reactors

4.1 Linking ROS and μ-ROS

Chapter 5

Tierless Haai

5.1 Haai to CHaai

Chapter 6

Evaluation

6.1 Evaluating Haai distribution

6.2 Evaluating Haai to CHaai

6.3 Evaluating CHaai on μ-Remus

Chapter 7

Conclusion & Future Work

Bibliography

[1] S. Van den Vonder, B. Oeyen, G. Salvaneschi, W. De Meuter and J. Nicolay, 'A survey on reactive programming and reactive streams,' *ACM Computing Surveys*, Nov. 2024, Manuscript submitted to ACM.

[2] E. Bainomugisha, A. Lombide Carreton, T. Van Cutsem, S. Mostinckx and W. De Meuter, 'A survey on reactive programming,' *ACM Comput. Surv.*, vol. 45, no. 4, Aug. 2013, ISSN: 0360-0300. DOI: 10.1145/2501654.2501666 [Online]. Available: https://doi.org/10.1145/2501654.2501666

[3] B. Oeyen, J. De Koster and W. De Meuter, 'Reactive programming without functions,' *The Art, Science, and Engineering of Programming*, vol. 8, no. 3, 11:1–11:30, 2024. DOI: 10.22152/programming-journal.org/2024/8/11

[4] A. A. Allaban, D. Bonnie, E. Knapp, P. Gokhale and T. Moulard, 'Developing production-grade applications with ros 2,' in *Robot Operating System (ROS): The Complete Reference (Volume 6)*, ser. Studies in Computational Intelligence, A. Koubaa, Ed., vol. 962, Cham, Switzerland: Springer Nature Switzerland AG, 2021, pp. 3–52, ISBN: 978-3-030-75471-6. DOI: 10.1007/978-3-030-75472-3_1

[5] H. Bruyninckx, 'Open robot control software: The OROCOS project,' in *Proceedings of the 2001 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 3, Seoul, South Korea: IEEE, 2001, pp. 2523–2528, ISBN: 0-7803-6475-9. DOI: 10.1109/ROBOT.2001.933002

[6] G. Metta, P. Fitzpatrick and L. Natale, 'YARP: Yet another robot platform,' *International Journal of Advanced Robotic Systems*, vol. 3, no. 1, pp. 43–48, 2006. DOI: 10.5772/5761

[7] B. Oeyen, J. Nicolay and W. De Meuter, 'A virtual machine for higher-order reactors,' in *Programming Companion 2024: Proceedings of the 8th International Conference on the Art, Science, and Engineering of Programming*, E. Soderberg and L. Church, Eds., New York, NY, USA: Association for Computing Machinery, 2024, pp. 52–56. DOI: 10.1145/3660829.3660840

[8] C. Descheemaeker, 'Compilation-based execution and optimisation of reactive programs on the bare metal: A vm approach,' Master's thesis, Vrije Universiteit Brussel, Brussels, Belgium, 2023.

[9] W. Shi, J. Cao, Q. Zhang, Y. Li and L. Xu, 'Edge computing: Vision and challenges,' *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016. DOI: 10.1109/JIOT.2016.2579198

[10] J. Gubbi, R. Buyya, S. Marusic and M. Palaniswami, 'Internet of things (iot): A vision, architectural elements, and future directions,' *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645–1660, 2013. DOI: 10.1016/j.future.2013.01.010

[11] J. Liu et al., 'Space-efficient trec for enabling deep learning on microcontrollers,' in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '23, New York, NY, USA: Association for Computing Machinery, Mar. 2023, pp. 644–659, ISBN: 9781450399180. DOI: 10.1145/3582016.3582062

[12] K. Sawada and T. Watanabe, 'Emfrp: A functional reactive programming language for small-scale embedded systems,' in *Proceedings of the 15th International Conference on Modularity Companion*, ser. MODULARITY '16 Companion, New York, NY, USA: Association for Computing Machinery, Mar. 2016, pp. 36–44, ISBN: 9781450340335. DOI: 10.1145/2892664.2892670

[13] K. Suzuki, K. Nagayama, K. Sawada and T. Watanabe, 'Cfrp: A functional reactive programming language for small-scale embedded systems,' in *Department of Computer Science, Tokyo Institute of Technology*, Tokyo, Japan.

[14] T. Hawkins, 'Controlling hybrid vehicles with haskell,' in *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming*, ser. ICFP '08, Victoria, BC, Canada: Association for Computing Machinery, 2008, ISBN: 9781595939197. DOI: 10.1145/1411204.2181025 [Online]. Available: https://doi.org/10.1145/1411204.2181025

[15] L. Pike, A. Goodloe, R. Morisset and S. Niller, 'Copilot: A hard real-time runtime monitor,' in *Runtime Verification*, ser. Lecture Notes in Computer Science, vol. 6418, Springer, 2010, pp. 345–359. DOI: 10.1007/978-3-644-16638-0_26

[16] M. Quigley et al., 'Ros: An open-source robot operating system,' in *IEEE International Conference on Robotics and Automation (ICRA) Workshop on Open Source Software*, vol. 3, 2009, p. 5.

[17] S. Macenski, T. Foote, B. Gerkey, C. Lalancette and W. Woodall, 'Robot operating system 2: Design, architecture, and uses in the wild,' *Science Robotics*, vol. 7, no. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074 [Online]. Available: https://www.science.org/doi/10.1126/scirobotics.abm6074

[18] K. Belsare et al., 'Micro-ros,' in *Robot Operating System (ROS): The Complete Reference (Volume 7)*, ser. Studies in Computational Intelligence, A. Koubaa, Ed., vol. 1051, Cham: Springer Nature Switzerland AG, 2023, pp. 3–55, ISBN: 978-3-031-09061-5. DOI: 10.1007/978-3-031-09062-2_2